ЗАНЯТИЕ 4 • AI-АГЕНТЫ

Kubernetes

Разворачиваем мультиагентную систему:
поды, сервисы, масштабирование и состояние агентов

Чернов Евгений
Telegram: @awe_eu

ГДЕ МЫ НАХОДИМСЯ

Программа курса: от первого агента до кластера

КУРС

Разработка и внедрение AI-агентов

1. LangChain + LangGraph: агент и граф
2. Интеграции: MCP, веб-чат, Telegram
3. Несколько агентов: A2A и Langfuse
4. **Kubernetes: агенты в кластере**

ЗАНЯТИЕ 4

От пода к мультиагентной системе

1–4 Основы на hello-agent

5 База в кластере: том и Job

6 Мультиагентная система

7 Масштабирование и состояние

ПОСТАНОВКА ЗАДАЧИ

Зачем нужен Kubernetes?

Представьте систему в проде, которой нужно постоянно обеспечивать четыре свойства:

1

Балансировка

Равномерно распределять нагрузку по нодам и репликам

2

Обновления

Выкатывать новые версии без простоя и с откатом

3

Отказоустойчивость

Перезапускать упавшие сервисы и держать нужное число реплик

4

Изоляция

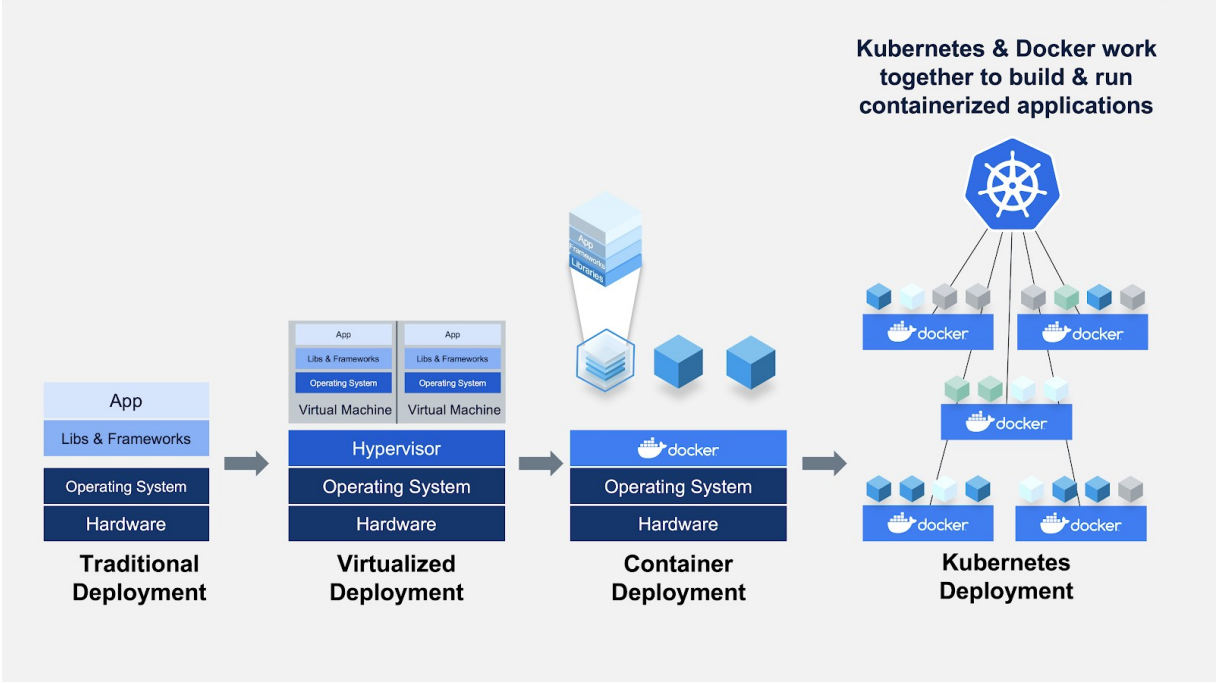
Разделять окружения и ресурсы между приложениями

!

А что умеет Docker? Только изоляцию. Балансировку, обновления и самовосстановление берёт на себя Kubernetes.

КОНТЕКСТ

От «железа» к оркестрации контейнеров



Traditional → Virtualized → Container → Kubernetes. Контейнеры дают лёгкую изоляцию поверх одной ОС, а Kubernetes добавляет управление их жизненным циклом на кластере машин.

ОПРЕДЕЛЕНИЕ

Что такое Kubernetes

Kubernetes (k8s) — система для автоматизации деплоя, масштабирования и управления контейнеризированными приложениями на кластере машин.

Поддерживаемые container runtime

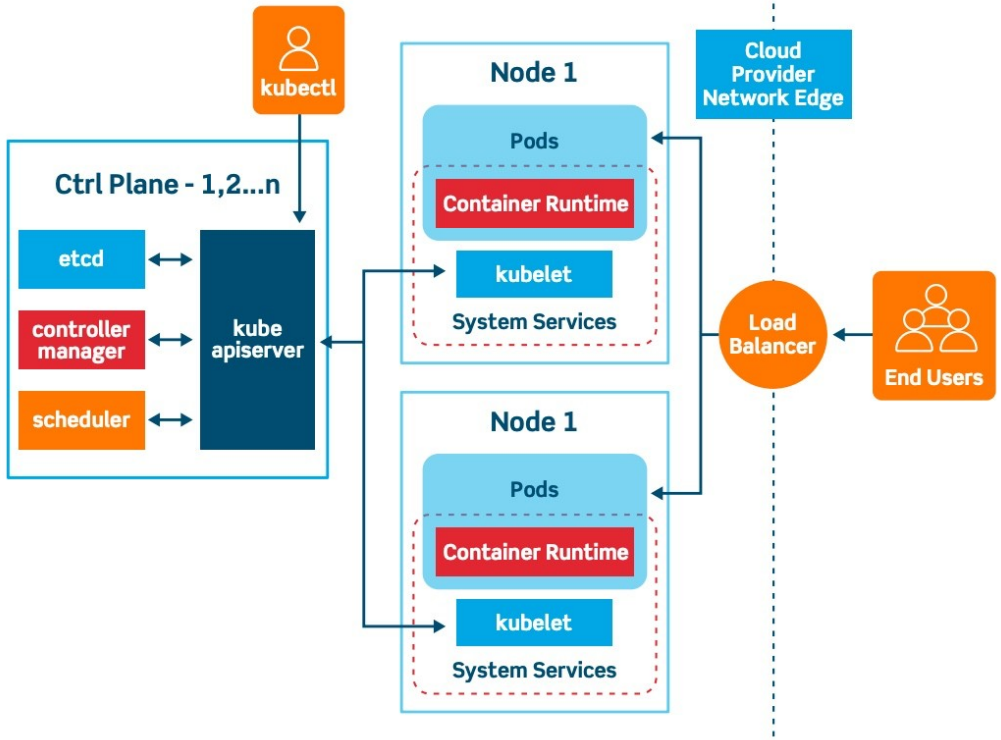
- **containerd** — стандарт де-факто, используется в k8s
- **CRI-O** — лёгкий runtime под Kubernetes CRI
- **Docker** — через shim; исторически самый известный
- **podman / rkt** — альтернативные движки контейнеров

Как запустить кластер

1. **Локально**
   minikube — учебный кластер на своём ноутбуке
2. **Лёгкий дистрибутив**
   k3s — минимальный k8s для edge и стендов
3. **В облаке**
   Yandex Cloud через terraform или меню администратора

АРХИТЕКТУРА

Как устроен кластер: Control Plane и рабочие ноды



Два уровня

- **Control Plane**
  «мозг» кластера: принимает решения, хранит состояние, планирует размещение подов
- **Worker Nodes**
  рабочие машины: на них реально запускаются контейнеры с приложениями
- **kubectl**
  CLI администратора — команды идут в kube-apiserver

КОМПОНЕНТЫ

Из чего собран кластер

CONTROL PLANE — управляющий слой

kube-apiserver

единая точка входа в кластер; через него идут все команды и запросы

etcd

распределённое хранилище состояния: что и где запущено

scheduler

решает, на какую ноду поставить новый под

controller-manager

следит, чтобы фактическое состояние сходилось с желаемым

WORKER NODE — рабочий слой

kubelet

агент ноды: запускает контейнеры и следит за их состоянием

container runtime

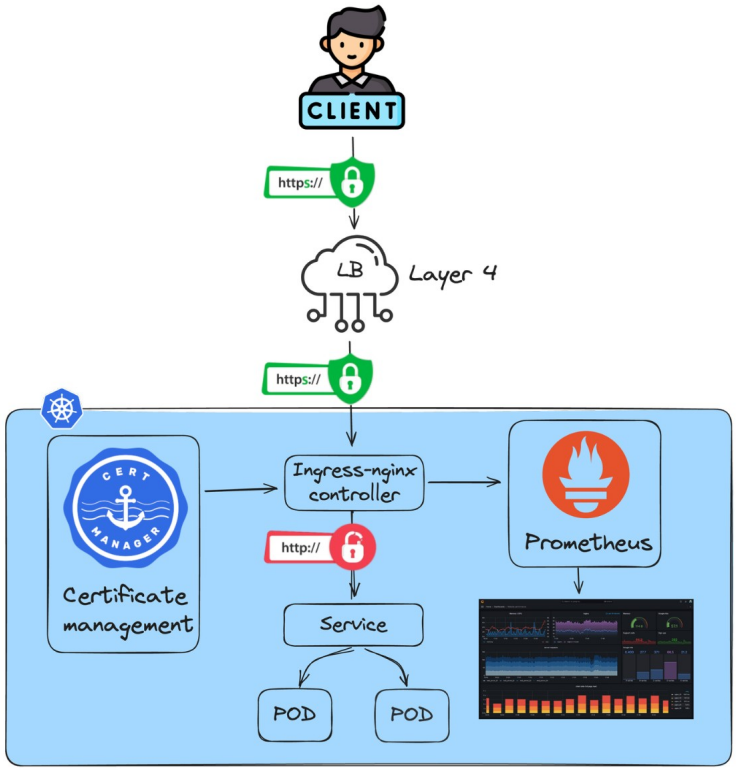
движок контейнеров (containerd / CRI-O), исполняет образы

kube-proxy

сетевые правила: маршрутизация трафика к подам сервиса

СЕТЕВОЙ ДОСТУП

Как клиент и админ достигают сервисов в кластере



Клиент → снаружи
Трафик идёт через Load Balancer (L4) на Ingress-контроллер, а тот — на нужный Service

Ingress
точка входа HTTP/HTTPS; терминирует TLS и роутит по хостам и путям

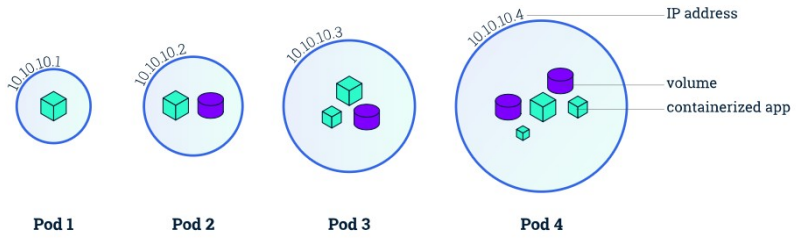
Админ → kubectl
команды идут в kube-apiserver, а не напрямую к подам

Мониторинг
Prometheus собирает метрики, Grafana их визуализирует

РЕСУРС · POD

Pod — атомарная единица запуска

Pod — наименьшая сущность в k8s. Это один или несколько контейнеров с общими сетью, IP-адресом и томами.



Каждый Pod получает собственный IP; контейнеры внутри общаются через localhost.

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: agent-pod-1
  labels:
    app: hello-agent
spec:
  containers:
  - name: agent-pod-1
    image: hello-agent:1
    resources:
      requests:  { cpu: 50m, memory: 32Mi }
      limits:    { cpu: 200m, memory: 64Mi }
    ports:
      - containerPort: 8080
```

РЕСУРС · REPLICASET

ReplicaSet — поддержание нужного числа реплик

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: hello-agent-rs
spec:
  replicas: 4
  selector:
    matchLabels:
      app: hello-agent
  template:
    metadata:
      labels: { app: hello-agent }
    spec:
      containers:
      - name: agent
        image: hello-agent:1
```

Что делает ReplicaSet

1. **Держит N подов**
   Всегда поддерживает заданное число реплик (replicas: 4)

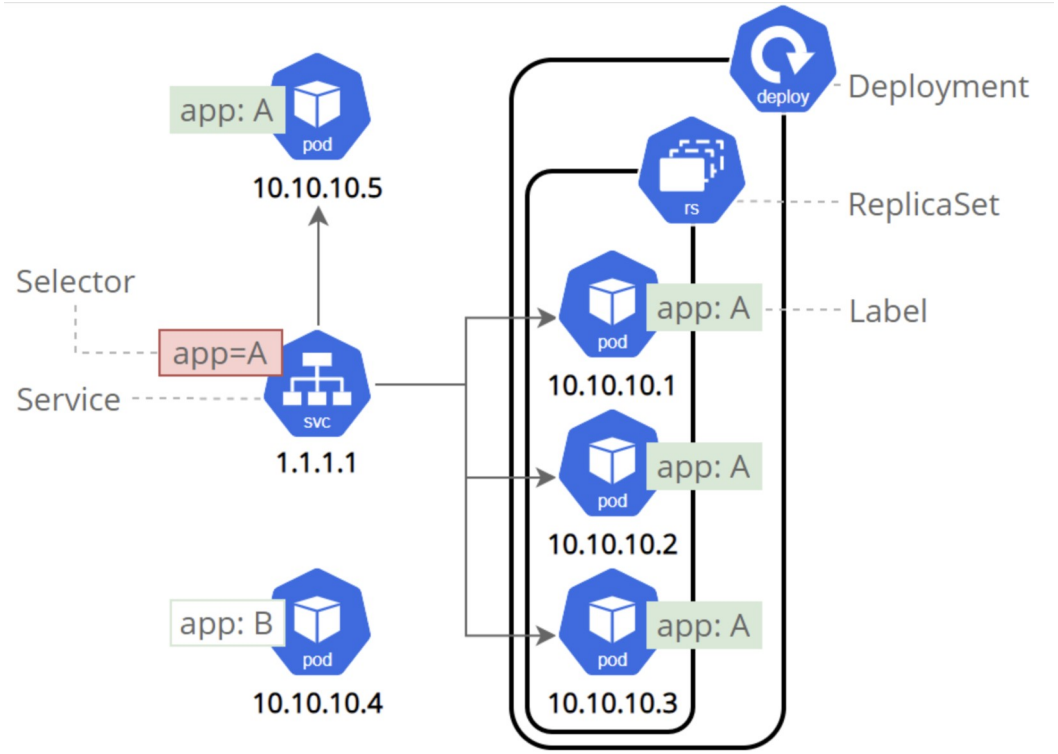
2. **Подбор по меткам**
   Управляет подами, чьи labels совпадают с selector

3. **Самовосстановление**
   Удалили под — RS тут же поднимает новый на замену

4. **Масштабирование**
   Меняем replicas в манифесте — число подов подстраивается

РЕСУРС · DEPLOYMENT

Deployment — управление состоянием приложения



Deployment → ReplicaSet → Pods

- **Декларативные обновления**
  задаём желаемый образ — k8s сам плавно заменяет поды
- **История ревизий**
  kubectl rollout history — видно все версии деплоя
- **Откат в один шаг**
  kubectl rollout undo возвращает предыдущую ревизию
- **RollingUpdate**
  maxUnavailable / maxSurge — обновление без простоя

РЕСУРС · DEPLOYMENT

Стратегия обновления и операции rollout

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: hello-agent }
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels: { app: hello-agent }
  template:
    metadata: { labels: { app: hello-agent } }
    spec:
      # nodeSelector: { load: high }
      containers:
      - { name: agent, image: hello-agent:1 }
```

rollout — управление выкаткой

```
$ kubectl rollout history deploy/hello-agent
$ kubectl apply -f deployment.yaml  # образ :2
$ kubectl rollout status deploy/hello-agent
$ kubectl rollout undo deploy/hello-agent   # откат
```

Размещение по нодам (nodeSelector)

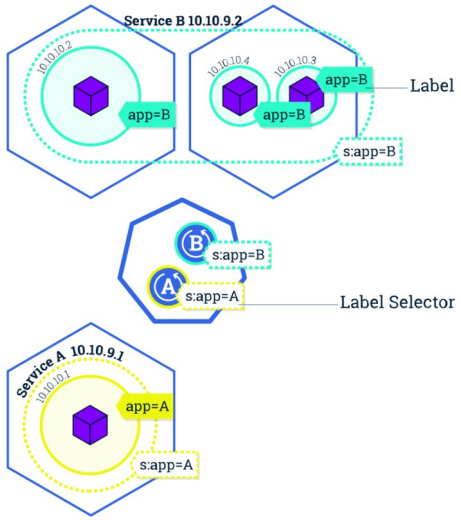
`kubectl label nodes minikube-m02 load=high`

Добавляем nodeSelector в манифест — и поды переезжают только на ноды с нужной меткой.

РЕСУРС · SERVICE

Service — стабильная точка доступа к подам

Service — логическая абстракция: единый IP и DNS-имя, за которыми через **label-селекторы** скрывается набор подов. Поды приходят и уходят — адрес Service остаётся постоянным.



Как работают селекторы

Под
несёт метки, например app: hello-agent.

Service
задаёт selector app: hello-agent и связывает все совпавшие поды.

Трафик
балансируется между ними автоматически.

РЕСУРС · SERVICE

Три типа Service

1

ClusterIP

ПО УМОЛЧАНИЮ

Внутренний IP, доступный только внутри кластера. Для связи сервисов между собой.

2

NodePort

ПОРТ НА НОДАХ

Открывает один порт на каждой ноде (30000–32767). Простой внешний доступ для стендов.

3

LoadBalancer

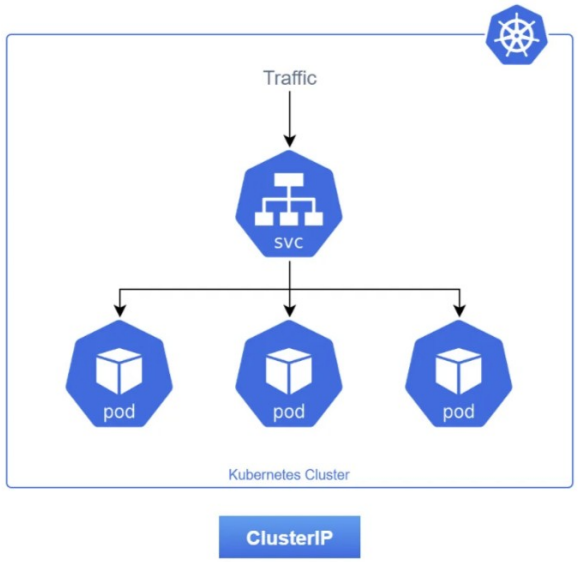
ТОЛЬКО В ОБЛАКЕ

Провайдер поднимает внешний балансировщик с публичным IP перед сервисом.

РЕСУРС · SERVICE

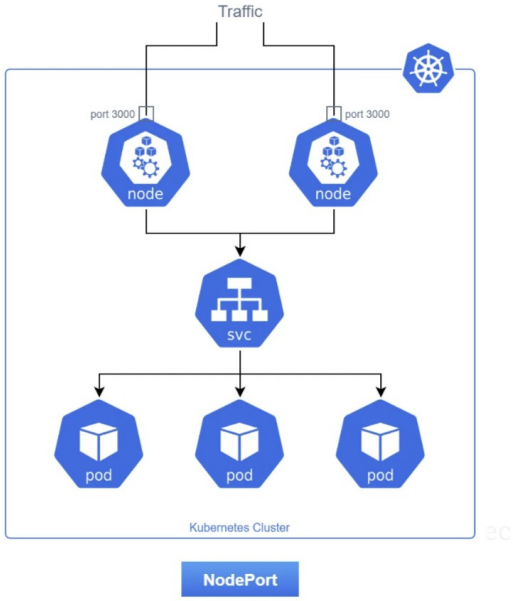
Три типа Service — наглядно

ClusterIP



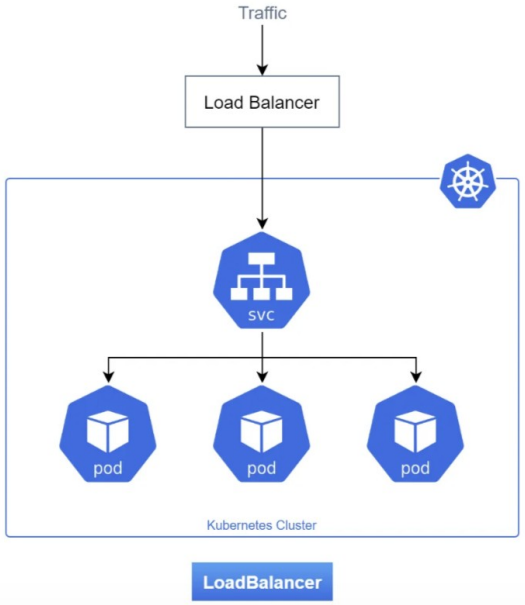
доступ внутри кластера

NodePort



порт на каждой ноде

LoadBalancer

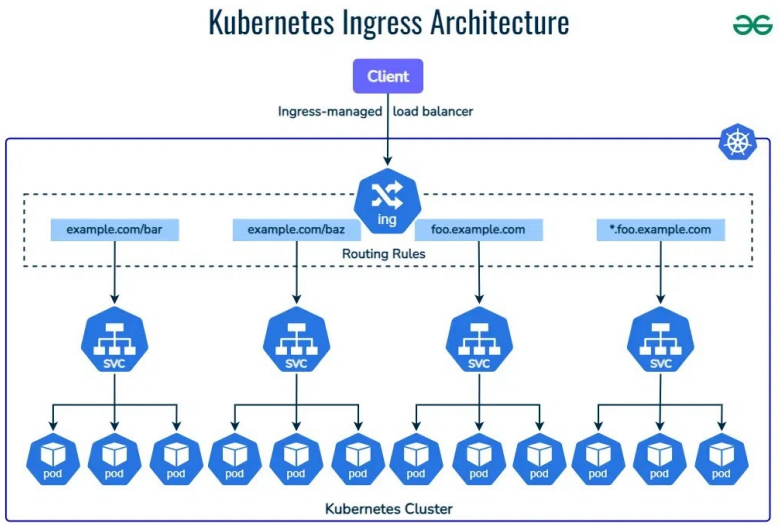


внешний IP из облака

РЕСУРС · INGRESS

Ingress — единая точка входа HTTP/HTTPS

Ingress — объект k8s API, который выставляет HTTP/HTTPS-роуты снаружи кластера и по хосту/пути направляет трафик на внутренние сервисы. Работает через Ingress-контроллер (например, nginx).



Зачем нужен

- Один вход вместо множества NodePort / LoadBalancer
- Роутинг по хостам: agents.test, risk.agents.test
- Роутинг по путям: /, /api, /metrics
- Терминация TLS в одном месте

КОНФИГУРАЦИЯ

ConfigMap и Secrets — отделяем настройки от кода

ConfigMap — несекретные настройки

configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: hello-agent-config }
data:
  config.yaml: |
    model: openai/gpt-4o-mini
    temperature: 0
```

Монтируется как файл: `volumeMounts → /etc/agent`
Внутри пода — обычный файл /etc/agent/config.yaml.

Secret — чувствительные данные

secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata: { name: hello-agent-secret }
type: Opaque
data:
  API_KEY: c2stZGVtbw==    # sk-demo   (base64)
  DB_PASSWORD: c2VjcmV0    # secret (base64)
```

Значения в base64, не шифрование: `echo -n sk-demo | base64`
Пробрасывается в под как переменные окружения через secretKeyRef.

НАБЛЮДАЕМОСТЬ

Мониторинг кластера: Prometheus + Grafana

1. **kube-prometheus-stack**
   Ставим Helm-чартом в namespace monitoring — сразу Prometheus, Grafana и экспортёры метрик

2. **Node Exporter**
   Автоматически встаёт на все ноды и отдаёт метрики хостов

3. **Grafana ↔ Prometheus**
   Grafana подключается к prometheus-operated:9090; дашборды кластера из коробки

4. **Свой дашборд**
   «Агенты в кластере»: память, CPU, реплики и перезапуски агентов

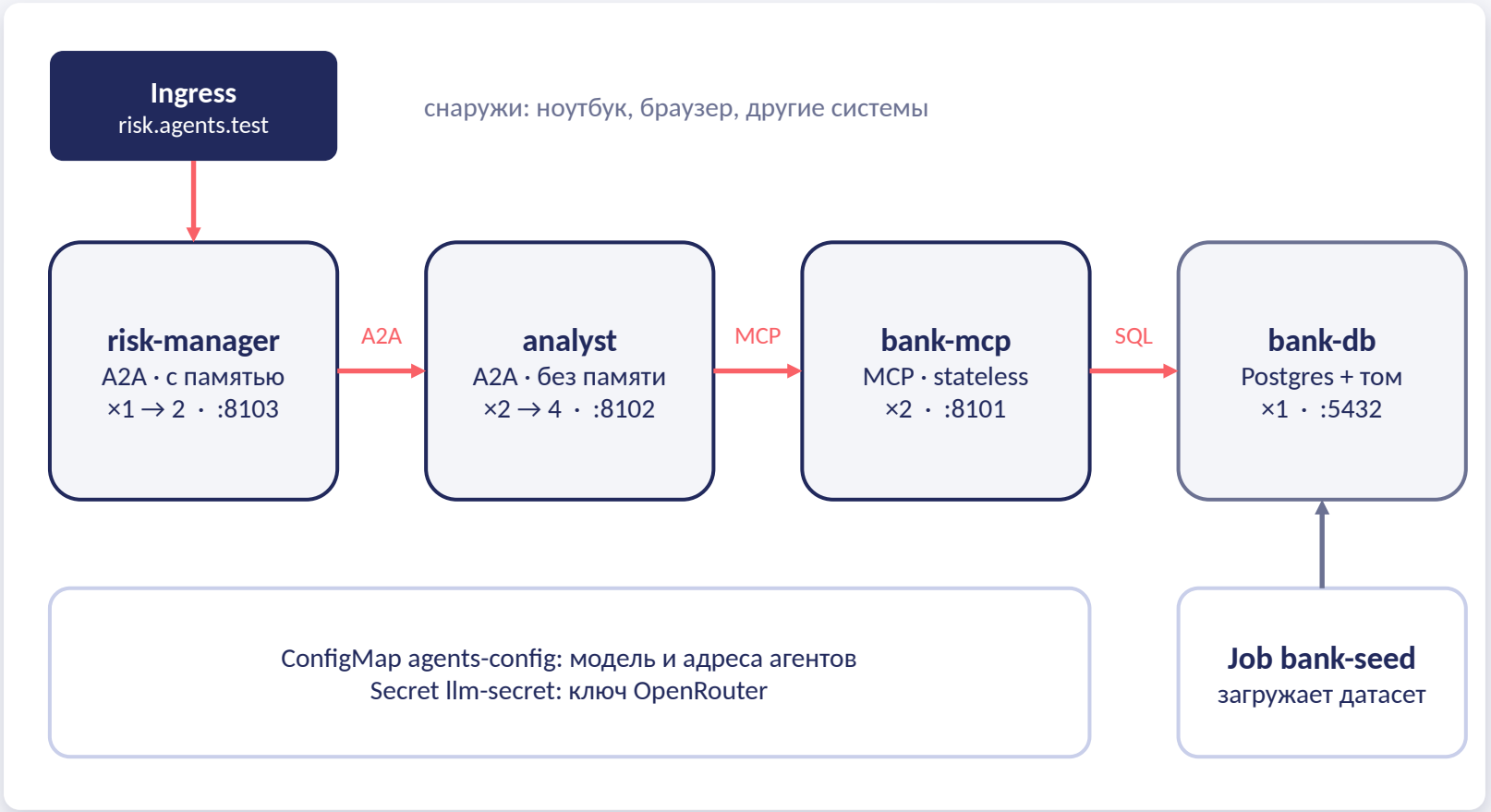
установка стека

```
$ kubectl create namespace monitoring
$ helm install prometheus \
    prometheus-community/kube-prometheus-stack \
    --namespace monitoring \
    -f part_9_monitoring/values.yaml \
    --version 65.1.1 --wait
```

Доступ через Ingress

`grafana.agents.test`
`prometheus.agents.test`
логин / пароль Grafana: admin / admin

Как наша мультиагентная система ложится в кластер



Что здесь важно

- **Снаружи — только риск-менеджер**
  Ingress risk.agents.test; аналитик и MCP видны лишь агентам
- **Адреса — имена Service**
  analyst:8102, bank-mcp:8101 — прописаны в ConfigMap
- **Один образ — разные роли**
  bank-agents:1, роль задаёт command: в манифесте

ГРАБЛИ

Что ломается, когда агентов становится много

Всё это мы поймали при подготовке занятия — на этом же кластере:

1

MCP-сессии

Сессия живёт в одной реплике: 18 из 20 вызовов падали. Лечит stateless_http

2

Память в поде

У каждой реплики своя память — разговор «теряется». Лечит Postgres

3

Долгие запросы

Под убивают через 30 с, а агент думает дольше — даём 120 с

4

CPU ≠ нагрузка

Агенты ждут LLM, CPU ≈ 0: HPA по CPU не поможет

!

Поды одноразовые. **Всё, что должно их пережить,** — память агентов, файлы, сессии — храним вне пода: в базе, на томе, в кэше.

ПРАКТИКА

Поднимаем учебный кластер minikube

cluster setup

```
# 3 ноды на QEMU (macOS), сеть — socket_vmnet
$ minikube start --driver qemu --network socket_vmnet \
    --nodes 3 --cpus 2 --memory 4g --disk-size 20g

# аддоны и образы агентов
$ minikube addons enable ingress
$ minikube addons enable metrics-server
$ minikube image build --all -t bank-agents:1 images/agents

# через Makefile проекта
$ make setup && make images
$ make hosts    # *.agents.test → IP кластера

# ключ LLM — в Secret, агенты — в кластер
$ make secret   # OPENROUTER_API_KEY из .env
$ kubectl apply -f part_6_agents/
$ kubectl get pods -o wide
```

Что происходит при setup

- **3 ноды** — один minikube эмулирует полноценный многонодовый кластер
- **ingress** — поднимается nginx Ingress-контроллер для внешнего доступа
- **metrics-server** — источник метрик CPU/RAM для kubectl top и автоскейла
- **/etc/hosts** — домены *.agents.test → IP minikube; с socket_vmnet туннель не нужен

ПРАКТИКА

Сценарий занятия: от пода к мультиагентной системе

Части 1–4	**Основы на hello-agent**	Поды и сеть, Service и Ingress, ReplicaSet и Deployment: самовосстановление, обновление v1 → v2 и откат, nodeSelector; ConfigMap и Secret
Часть 5	**База в кластере**	PersistentVolumeClaim и Postgres, init.sql из ConfigMap, Job загрузки датасета с initContainer — данные переживают перезапуск пода
Часть 6	**Мультиагентная система**	MCP-сервер, аналитик и риск-менеджер: Deployment + Service, startup/readiness/liveness-пробы; снаружи — только риск-менеджер
Часть 7	**Масштабирование и состояние**	4 аналитика одной командой; риск-менеджер на 2 репликах «забывает» — память в Postgres. Бонус: CronJob, HPA, DaemonSet, Grafana

МАНИФЕСТЫ? МАНИФЕСТЫ!

Live demo

Разбираем YAML-манифесты и применяем их в кластере, шаг за шагом

- `kubectl apply -f .`
- `kubectl get pods -o wide`
- `kubectl logs -f -l app=analyst`
- `kubectl scale … --replicas=4`

ИТОГИ

Итоги: Kubernetes для агентов

Deployment

агент = Deployment: реплики, самовосстановление, обновление и откат

Service

постоянный адрес и балансировка: агенты находят друг друга по имени

Ingress

вход снаружи — только для агентов, которым он нужен

ConfigMap / Secret

модель и адреса — в ConfigMap, ключ LLM — в Secret, а не в образе

PVC · Job

данные на томе переживают под; Job — разовые задачи вроде загрузки

Состояние — вне пода

память агентов — в базе: тогда агентов можно масштабировать